

# Warehouse pick and place for tightly packed shelves

## JSAN

Simone Cosentino  
sicos23@student.sdu.dk

Adam Kolodziej  
adkol23@student.sdu.dk

Jonas Mariager Jakobsen  
jonja20@student.sdu.dk

Nina Siwakoti  
nisiw23@student.sdu.dk

**Abstract**—This project addresses the industrial challenge presented by the Schmalz initiative: a solution integrating vacuum gripping technology with drones for payload pick-and-place operations. The objective is to design and prototype a drone that can autonomously retrieve and deliver boxes from shelves to defined landing points with as little human intervention as possible. Traditional handling methods are quite challenged in environments where boxes are stored in vertical stacks and space is limited, especially when retrieving boxes from hard-to-reach locations. The proposed prototype solution is based on a drone equipped with a suction gripper that can grasp boxes from the side and an active balancing system, enabled by an actuated counterweight, to stabilize flight in tight spaces. A centralized database with unique location identifiers for each box is maintained where users can search, select, and retrieve boxes through a UI.

**Index Terms**—warehouse, autonomous drones, pick-and-place operations, suction gripper mechanism, dynamic warehouse systems, shelf-level navigation, box retrieval systems, interfacing ROS and web applications, industry 4.0 technologies

### I. DESCRIPTION OF THE INDUSTRIAL CHALLENGE

The industrial challenge addressed by this project stems from Schmalz’s interest in leveraging drone technology to enhance warehouse automation. The core of this challenge involves the design and development of an advanced autonomous drone system capable of autonomously retrieving and delivering specific boxes from shelves to designated landing points. This system aims to tackle critical issues in warehouse logistics, such as reducing manual labor, improving efficiency, and enhancing accuracy in operations.

Warehouse environments often involve large volumes of boxes arranged in vertical stacks, with limited space for conventional handling methods. Traditional systems struggle to effectively manage these environments, particularly when dealing with boxes stored at the top of stacks or in areas that are difficult to access. The proposed solution involves a drone equipped with a suction gripper and a tray to handle boxes by grasping them from the side. The drone’s active balancing system, aided by an actuated counterweight, ensures flight stability even when navigating narrow spaces between shelves.

This technology targets warehouses where multiple boxes are arranged in a structured system, with units, shelves, and stacks. The boxes are assigned unique location identifiers stored in a database, which is sent to the drone’s navigation system for precise autonomous box retrieval. The system is designed to work with limited human interaction, improving efficiency and reducing errors in warehouse management.

The significance of this challenge is amplified by the complexities inherent in the project. Schmalz, acting as both supplier and client, has yet to fully approve the final solution, making the project highly dynamic. There are challenges in defining clear acceptance criteria, securing adequate resources, and managing the evolving requirements of the project. Additionally, the absence of an established supply chain and Schmalz’s limited experience with drone systems introduces additional hurdles. Despite these challenges, the project represents an opportunity to lay the foundation for future advancements in warehouse automation, with the potential to create scalable, cost-effective solutions that improve space efficiency, reduce labor costs, and enhance safety in logistics operations.

### II. STATE OF THE ART

The increasing demand for automating goods transport through pick-and-place operations using drones is mainly driven by improved efficiency and cost-effectiveness compared to traditional methods. In this regard, several studies have been conducted to investigate the advantages of using drones alongside humans for warehouse automation. Semi-automated drones are integrated to increase the efficiency in warehouse inventory management in the study [8]. It is concluded that efficiency in warehouses is increased by resolving discrepancies through frequent cycle counting without disrupting processes, reducing manual labor, improving record-keeping, and leading to smoother logistics and better inventory management.

The drone system improves efficiency in pick-and-place operations by autonomously identifying, retrieving, and accurately positioning objects within a specified area. It also demonstrates significant improvements in operational speed and accuracy compared to manual methods, minimizes human intervention, reduces labor costs, and mitigates the risk of human error [2]. Similarly, human-drone interactions in warehouses are very effective, as demonstrated in [9] with respect to both pick-and-place operation and human safety working alongside drones.

### III. APPROACH

#### A. Software

The software architecture depicted in Figure 1 integrates cutting-edge technologies to ensure effective communication between system components, optimizing overall functionality. This system constitutes the central interface through which both many operators and drone systems communicate through.

Key components include the React frontend, the Robot Operating System (ROS2 Humble [3], referred to as ROS throughout this report) interface, and the PostgreSQL database, which collectively provide a user-friendly and efficient experience for warehouse workers. Leveraging this system is the software controlling the drone. This software includes a ROS node for online control of the drone, as well as a trio of ROS nodes that handle the interaction between the hardware components and the control software responsible for picking and placing packages.

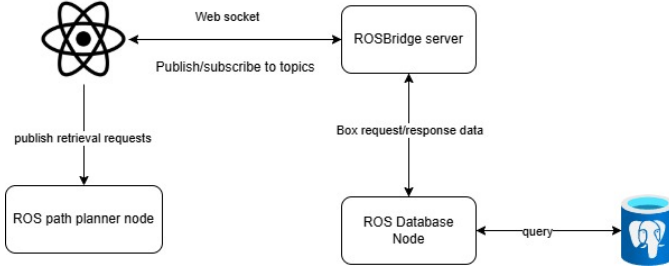


Fig. 1: React app - ROS - database workflow

The React application serves as the central interface, allowing users to interact with the system. Through a WebSocket connection to the ROSBridge server, the app provides real-time updates, such as results from box retrieval requests. This WebSocket connection enables fast and reliable [1] data exchange between the frontend and the ROS components.

The ROSBridge server acts as a middleware, translating WebSocket messages from the React app into a format that ROS understands and vice versa. This enables smooth communication between the two systems. [5]

The ROS database node is responsible for handling incoming box retrieval requests. It queries the PostgreSQL database using filters for box locations, returning matching results in real-time. These results are then published to a dedicated ROS topic, which the React app subscribes to, ensuring users receive the latest data dynamically.

The PostgreSQL database stores all relevant information about box types, as well as dimensions and locations of stacks, shelves and units. It is structured to reflect the layout of a physical warehouse as shown in Figure 2. Each unit contains multiple shelves, each shelf holds stacks, and within each stack, there are individual boxes. These boxes are assigned a specific type, carrying information such as dimensions and weight.

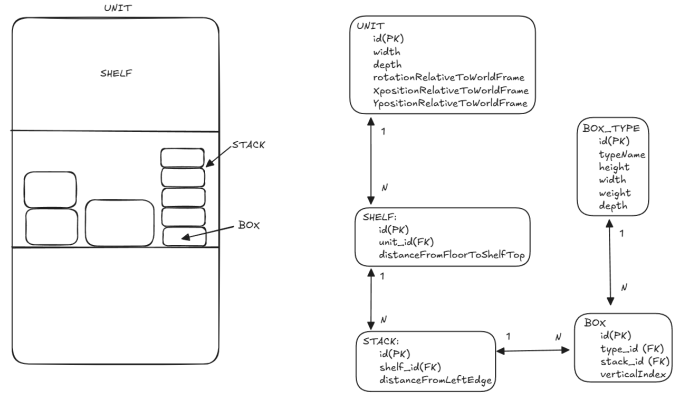


Fig. 2: DB E-R diagram

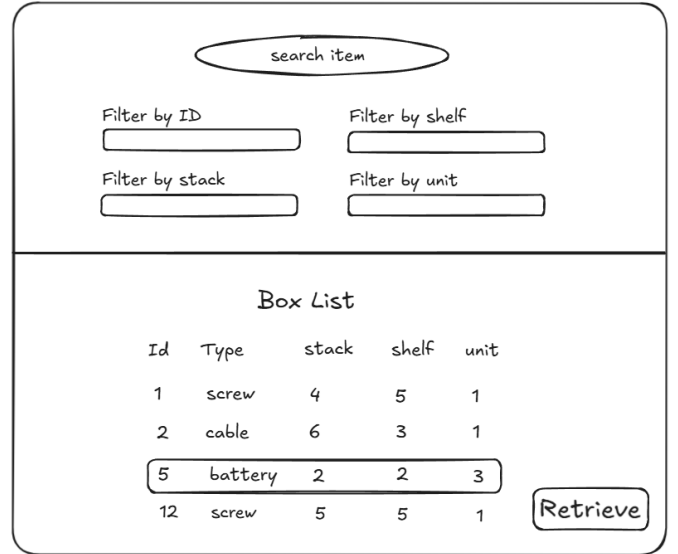


Fig. 3: UI mock-up

#### Key Differentiators:

- **Robust communication:** WebSocket communication between the React app and ROSBridge ensures low-latency, bidirectional data exchange. Unlike traditional HTTP, which is request-response-based, WebSockets allow for real-time updates, meaning the server can send data to the client without waiting for a client request. [10]
- **Scalability and flexibility:** The system is designed to scale. ROS allows the easy integration of additional drones or storage units without major architectural changes, providing room for future expansion, such as the integration of more sensors or advanced filtering capabilities. [7]
- **Cost-effectiveness:** The use of widely-adopted, open-source technologies—namely ROS, React, and PostgreSQL—reduces reliance on proprietary systems, keeping operational costs low while maintaining high functionality.
- **User-friendly interface:** The React frontend is designed with simplicity and usability in mind, offering an intuitive

and interactive interface, as depicted in Figure 3. Users can easily request box information and receive real-time updates with minimal setup, ensuring accessibility for a diverse range of users, from operators to administrators.

By leveraging ROS for communication, PostgreSQL for data management, and React for the frontend interface, the system represents a highly robust and adaptable solution for autonomous warehouse automation. [6]

1) *React App*: The React application serves as the user interface through which warehouse personnel interact with the autonomous drone system. It enables users to search for boxes, request their retrieval, and receive visual feedback regarding the system's actions.

a) *State Management*: The React app uses several states to manage its data. The `filters` state stores the search criteria, including `boxName`, `boxId`, `stackId`, `shelfId`, and `unitId`. These criteria are essential for defining the search query. The `boxInfo` state holds the list of boxes that match the applied filters, while `selectedBox` tracks the box chosen by the user for retrieval. Additionally, the `retrieveMessageSent` state indicates whether a retrieval message has been sent to the ROS system.

b) *ROS Integration*: To integrate with ROS, the React app uses the `ROSLIB` JavaScript library [4]. This library establishes a `WebSocket` connection to the ROS master via `roslib.Ros`. The frontend subscribes to the `/database/box_info` topic to receive real-time updates on box data based on the user's search criteria. When a user initiates a search or requests box retrieval, the app publishes messages to the `/database/box_requests` and `/path_planner/retrieve_box` topics, respectively.

c) *Functionality*: When the user inputs search criteria and presses the `Search` button, the app sends a request to ROS to retrieve the corresponding box data. Upon selecting a box and pressing the `Retrieve` button, a retrieval request is sent to ROS. After the retrieval message is sent, the app displays a confirmation message to the user for two seconds, notifying them of the action's success.

2) *Database ROS Node*: The database ROS node manages communication between the ROS system and the PostgreSQL database. It listens for retrieval requests from the frontend and queries the database to retrieve relevant box information.

a) *Database Connection*: The node connects to the PostgreSQL database using `SQLAlchemy`, which facilitates interaction with the database. The database contains several tables, including `Box`, `BoxType`, `Shelf`, `Stack`, and `Unit`, which store the necessary data about the boxes. The node interacts with the database through a session, generating SQL queries to retrieve data from these tables.

b) *Box Request Handling*: Upon receiving a box retrieval request, the node extracts the filters from the incoming message. It constructs an SQL query to search the database based on the provided filters, such as `boxName`, `boxId`, `stackId`, and others. The query is executed, and the resulting box data

is published to a ROS topic, which is then subscribed to by the frontend.

c) *Example SQL Query*: The query generated by the node uses `JOIN` operations to filter boxes based on multiple criteria. Below is a simplified example of the query:

Listing 1: SQLAlchemy Query Example

```
query = session.query(Box).join(Box.box_type).
    join(Box.stack).join(Stack.shelf).join(
        Shelf.unit)
if box_name:
    query = query.filter(Box.box_type.has(
        BoxType.typeName.ilike(f' %{box_name}%'
        )))
if box_id:
    query = query.filter(Box.id == box_id)
```

3) *Path Planner*: The path planner node is responsible for handling box retrieval commands from the frontend and planning the movement of the drone to retrieve the specified box.

a) *Subscription to `/path_planner/retrieve_box`*: The path planner listens for retrieval requests on the `/path_planner/retrieve_box` topic. When a request is received, it processes the associated box data, which includes information about the box type, stack, shelf, and unit. This information is essential for the path planner to calculate the most efficient route for the drone.

b) *Fetching Unit and Box Data*: To generate the necessary movement plans, the path planner fetches additional data about the box, stack, shelf, and unit from the database. Helper functions such as `fetch_unit_data`, `fetch_box_data`, `fetch_shelf_data`, and `fetch_stack_data` are used to retrieve this data, ensuring that the drone can navigate the warehouse efficiently and accurately.

c) *Movement calculation*: Once the necessary data is retrieved, the path planner computes waypoints for the drone to reach the box. These waypoints include the travel time between each waypoint and are calculated based on a desired velocity. Although the movement logic is not fully implemented, it involves calculating the best path from the drone's position to the box that is to be retrieved.

4) *Offboard position controller node*: Once the waypoints have been planned and transferred to the drone through the ROS network, the drone converts these waypoints into a trajectory in real time via linear interpolation, making use of the duration included with each waypoint. Each waypoint can have one of three types, basic waypoint movement or a box interaction, either pick or place. This functionality is implemented in the ROS node `offboard_ctrl_by_request`.

a) *Waypoint-type based state machine*: The basic waypoint based movement does interpolation by using two waypoints at once to generate the position commands on a straight line between the points. Once the path between the waypoints has been traversed the waypoint which was traveled from is switched out with the next so the process can repeat. This means the original path could potentially be interrupted

smoothly by replacing the upcoming waypoints, in a more dynamic warehouse with multiple drones.

*b) pick and place command:* The two pick and place waypoint types are implemented as a state machine to enforce a strict sequence of actions. That is in order: Orient the tray towards the destination that is about to be picked from or placed to, slowly approach while keeping the position steady, send the command to the grabbing system ROS node, let the grabbing system handle the pick or place operation waiting for a response, move away from the shelf back to the original position. The drone is then able to continue with the next waypoint and so on.

*5) Box grabbing system:* In terms of software the box grabbing system consists of 3 ROS nodes, two of which are for interfacing with hardware. The specific hardware solution is more fully described in Section III-B.

*a) Grabbing node:* The `main_grabbing_node` is responsible for handling the sequence of actions required to pick and place packages. The node functions through a state machine, which is triggered by a simple command telling it to either pick a package or place the package that it is currently holding onto as mentioned in the previous Section III-A4b. The `main_grabbing_node` itself is responsible for coordinating 4 elements: Two trolleys, one for the suction cup and one for the counterweight, the centering mechanism and the suction pump itself. The Order of operations for the two actions, pick and place, is showcased in this video: <https://youtu.be/YygJES3m-ps>.

*b) Multi motor node:* The `multi_motor_node` interfaces with the Dynamixel motors used through a USB connection to the Dynamixel U2D2 as shown in Figure 7. This node handles all the motors at once by providing a generic interface consisting of a motor id and an arbitrary command. The node allows configuration of an any number of Dynamixel motors with one of 4 operating modes: two-position mode, differential mode, centering mode, or absolute position mode. Two position mode is used for the trolley with the suction cup to go between its preconfigured retracted and extended position on command. Differential mode is used for the control loop of the adjustable counterweight which takes a desired velocity as input. The centering mode is a hybrid between two position mode and differential mode, it extends to a set position like two-position mode but retracts with a specific velocity checking the current to stop when it is firmly holding onto the package where it continues to apply force until directed to extend again. The absolute position mode was used for testing purposes as described in Section V-A1.

*c) Pump node:* The `pump_node` is responsible for interfacing with the Schmalz vacuum pump through the I/O pins of the raspberry pi 4. The node handles the 3 pins for "Suction" and "blow off" and "part detected" these 3 pins do not go directly to the pump, but instead through the 2 circuits described in III-B2c. This suction-pump hardware interface is controlled through a single ROS topic giving status feedback on another topic.

*6) Counterweight balancing:* As part of the `main_grabbing_node` a control loop actuates the counterweight motor to balance the drone minimizing the difference between the output level of each motor. This is done to ensure that packages can have different weights with a minimal impact on the stability of the drone during flight. The control loop uses the immediate difference in control levels between the two front and back propellers to create a heuristic for the imbalance in the system. This heuristic is passed through a simple gain controller with a deadzone to eliminate possible instabilities. The controller is shown in Equation 1, where  $c$  is the deadzone,  $h$  is the heuristic and  $k$  is the gain.

$$u = \begin{cases} k \cdot (h - c) & h > c \\ k \cdot (h + c) & h < -c \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

## B. Hardware

*1) Mechanical design:* As a drone platform, the SDU drone was used. To perform the required procedure of grabbing packages from the side, the UAV had to be modified and equipped with necessary components such as the Schmalz pneumatic system. First, the drone was partly disassembled and measured. Then the design in CAD software was conducted. The idea was to have a single track with two trolleys moving on it, one with a counterweight and one with a suction cup. To limit weight and keep everything stiff, a 15x27mm wooden beam was used as a track support. 3D printing was used not to extend the budget. The gear track was made with a Herringbone shape to make it self-centering and work as smoothly as possible.



Fig. 4: CAD design

The second part of the hardware modifications was a tray for boxes with a centering mechanism. The construction has to be long enough to be able to grab the box from the side so as to keep the propellers a safe distance from the boxes and shelves the issue can be visualized via Figure 6. To reduce the weight 9x9mm wooden rods were used. The centering mechanism was, on the other hand, fully 3D printed. The following video clearly shows how the tray was designed: <https://youtu.be/XpcWy9O8dFc>.

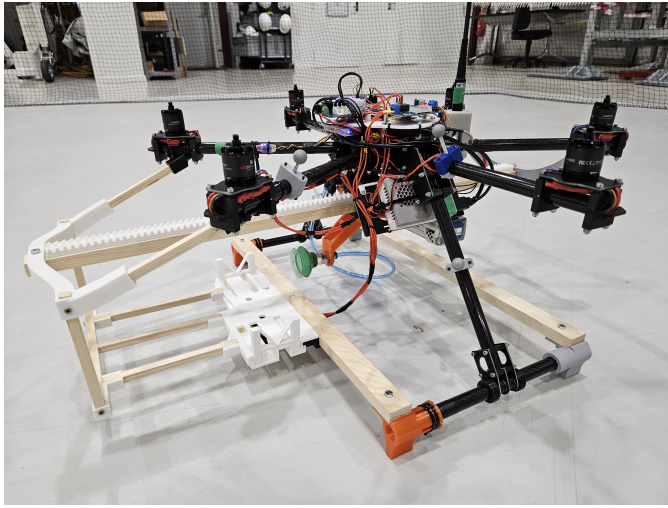


Fig. 5: Assembled drone

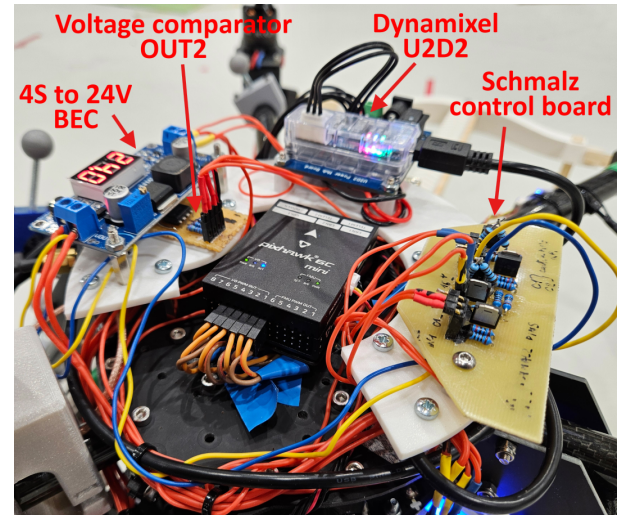
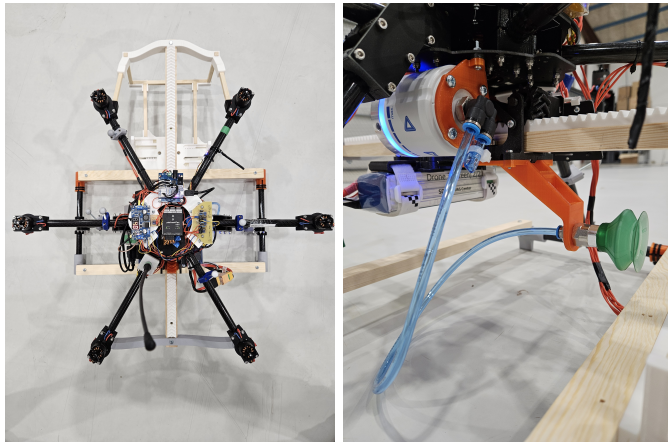


Fig. 7: Electronics mounted on the drone



(a) Top view

(b) Schmalz equipment

Fig. 6: More photos of the prototype

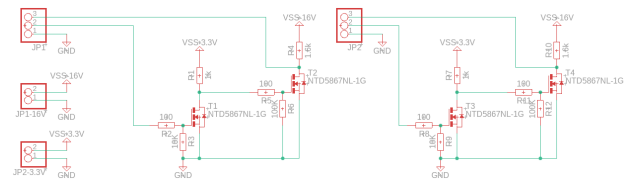
All the mentioned mechanisms were powered with XL330-M288-T Dynamixel motors. Calculations were done to find the balance between the motors' strength and mass. XL330-M288-T is run with 5V; the maximum current on this voltage is 1,47A. Under these conditions, their stall torque is 0,52Nm. They also provide feedback data like current, position, velocity, etc., which is crucial for this project—it is used throughout the control procedure. The components - if possible - were placed to help keep the center of mass in the middle of the UAV. For instance, the Schmalz pump and RPi were attached exactly on the opposite sides of the drone. Figures 4 and 5 shows a comparison between design and reality.

2) *Electronics*: All of the PCBs (except RPi) are mounted on top of the drone as seen in Figure 7.

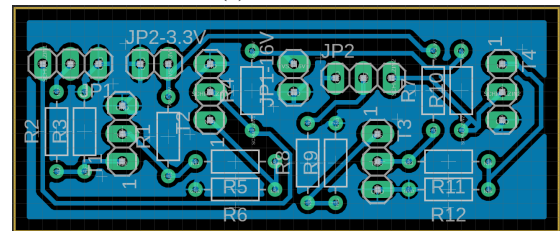
a) *Raspberry Pi 4*: As a companion computer Raspberry Pi 4 was used. It is a fairly powerful unit which was enough for this project. If it was necessary to perform much more intensive calculations or run an AI, it would be required to consider better units.

b) *BECs*: Power management on the UAV was essential and not so simple in this case. The Schmalz Electrical Vacuum Generator ECBPMi operates on 24V. A step-up voltage converter was used to go from a 4S battery. On the other hand, motors running both trolleys and centering mechanism, run on 5V so step-down BEC was a must-have as well.

c) *Operating on Schmalz equipment*: To run the system including Schmalz pneumatic equipment and motors, additional electronic components had to be designed and manufactured. PCBs were mounted on top of the drone.



(a) Schematic



(b) Board

Fig. 8: Custom PCB to control ECBPMi

#### Control board

The most important board, without which Schmalz equipment wouldn't work at all, is a custom-made PCB operating on the IN1 and IN2 inputs of the Electrical Vacuum Generator ECBPMi. The design of the board is showcased in Figure 8. IN1 is the "Suction" signal input and IN2 is the "Blow off" signal input. Both of them are activated with a voltage over

15V. RPi which controls the logic operates on 3.3V. To be able to turn the pump on, the 3.3V control signal must be converted to more than 15V. To achieve that, a custom control board was developed. It utilizes MOSFET N-channel transistors.

#### *Comparator*

Similar to the *Control Board*, OUT2 (Signal output for “Part Present”) output signal of the Electrical Vacuum Generator ECBPMi had to also be converted - this time from 24V to 3.3V. To achieve that simple voltage comparator was used. As a reference voltage, battery 4S voltage was taken. When something is grabbed by the suction cup, the signal is converted through the comparator to the RPi.

#### *C. First fly, encountered issues and solutions*

The UAV mass with one 6250 mAh LiPo battery was 4.410kg. It is worth mentioning that the drone was supposed to use two batteries (and ground tests were performed with two battery setups).

The first flight is presented in this video: <https://www.youtube.com/watch?v=inSqsNFQjJI>. The drone turned out to be much too heavy to use during the missions. It doesn't have enough power (over 80% throttle to take take off). The immediate solution for that would be using more powerful motors and ESCs (original are 20A). There was a plan to equip it with Emax ECO II Series 2807 3-6S 1300KV motors (since one of the team members has them available) with 50A ESC but due to time restrictions and the fact that modifications of the drone platform to such an extend are not necessarily allowed, the decision was made to perform ground tests instead.

### IV. EXPERIMENTS

#### *A. Hardware prototype*

Due to encountered issues explained in III-C, all of the tests were designed to be done on the ground.

1) *Thrust test*: This test's purpose was to verify the influence of transported packages on the generated thrust. Packages should decrease the thrust by creating a disturbance of the airflow. To experiment, a scale was used (which can measure up to 30kg). The drone was placed on the scale with an additional weight to keep it in place. The scale was tared so only thrust value was displayed. Three series of data were collected - each series with different size of package sizes: 70x109x122mm, 67.5x111x182mm, and 89x146x177mm. During the test position of the package was changed. The ROS node was created to enable easy and fast operation of the system during the test. To move the trolley with a package attached, commands were manually sent to set the position of the trolley to 1 of 11 possible positions, from fully retracted to fully extended linearly spaced along the track. The test is presented in the following video: [https://youtu.be/3S\\_gwiJq1sI](https://youtu.be/3S_gwiJq1sI).

The test was conducted with 50% of throttle in the Manual mode. Without any package, 24.6N of thrust was measured. Packages sizes: 70x109x122mm, 67.5x111x182mm, 89x146x177mm.

2) *Counterweight test*: This test required the design and construction of an additional platform with a ball joint. Since the counterweight system is based on the control signals for motors, it was a must-have to arm the drone and make it “flying”. To fulfill this requirement the drone was attached to the pole mounted on the wide platform to make the construction stable. The attachment point was made as a 3D printed ball joint as showcased in Figure 9. The following video: <https://youtube.com/shorts/9BHwiUSnsf4> shows how smooth the joint turned out to be. This kind of joint allows flight simulation of the drone - it has to stabilize itself and it can do roll, pitch, and yaw movements.

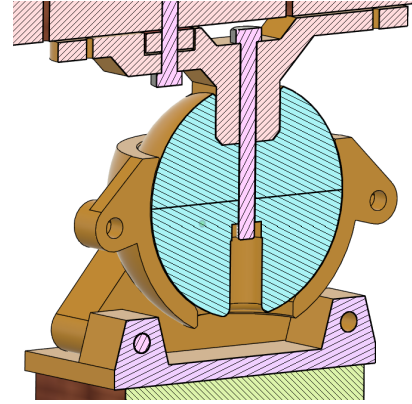


Fig. 9: Ball joint design

For the test, the drone was equipped with two 6250mAh batteries (2x563g=1126g) and a 285g package. “Flights” were performed in the stabilized mode. PID tunes were changed to make the drone more stable. Nevertheless, it was necessary to increase the throttle to 60/70% to level the drone. Then the throttle was reduced to 50/60%. During the test, the package was moved back and forth on the tray to see if the counterweight automatically adjusts its position. The video shows how the system works in real life: <https://youtu.be/P92HUGDOUhc?feature=shared>.

#### *B. Software prototype*

A link to a YouTube demo of our user interface is provided, demonstrating a straightforward use case in which the user searches for an item and receives a list of boxes containing that item. The demo highlights the functionality of filters, dynamic search results, and the retrieval button. You can view the demo at: <https://youtu.be/3EaYNRA8Ghs>

1) *System Response Time Evaluation*: The purpose of this test was to assess the system's response time during a box retrieval operation. The test aimed to measure the time taken from sending the retrieval request, which includes the relevant box data, to receiving the corresponding response from the backend system. Two different tests were performed to calculate the response time. The console time which is the time between sending the request and receiving the response at the frontend and the retrieve operation time which is the time

the backend takes to process and complete the request that includes everything from receiving the request to performing any database lookups and calculations are measured.

The test involved creating the ROS topic `'/path_planner/retrieve_response'` to publish a message containing the result of the operation. In the frontend, the user initiates the retrieval process, when they click on the **"Retrieve"** button, sending a message with the box data to the `'/path_planner/retrieve_box'` topic. The backend processed the request, and once the operation was completed, a message containing the response time was published to the `'/path_planner/retrieve_response'` topic.

Similarly, on the frontend, JavaScript's `'console.time'` and `'console.timeEnd'` functions were used to log the duration of the retrieval request which allowed us to measure the time it took to send the request and receive the response. This time measures from when the request is sent (from the frontend) to when the response or action was completed (in the backend). The time is printed as a log message in the console on the frontend. The results were then analyzed to determine the system's responsiveness, highlighting any delays or performance issues during typical box retrieval operations.

## V. RESULTS

### A. Hardware

1) *Thrust test:* The purpose of the thrust test was to verify the influence of transported packages on the generated thrust. Packages should decrease the thrust by creating a disturbance of the airflow. It was proved that the packages have a significant influence. The graph presented in Figure 10 shows how the thrust changes during the grabbing procedure. 10 positions were tested for 3 different sizes of packages. Initially, the thrust value is the biggest because the package is the closest to the drone's body - it is the furthest point from the propellers. While extending the suction cup, the thrust is reduced because the package disturbs the airflow from the propellers. From about the 5th position, the thrust increases again. The package is moved out of the airflow of the propellers. The largest decrease in thrust from normal was  $24.6N - 20.8N = 3.8N$  or 15%.

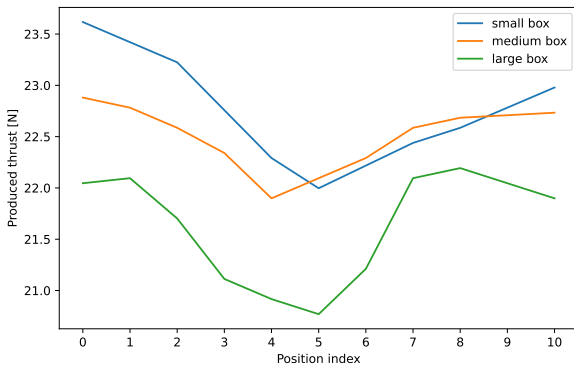


Fig. 10: Produced thrust in relation with position index

2) *Counterweight test:* First of all, the drone was struggling to stabilize which can be seen in the provided video. Again, the platform has to be more powerful. In spite of this, the system was proven to work properly. As the graph below presents, when the position of the suction cup was changed, the velocity of the counterweight motor reacted accordingly. As can be seen in Figure 11 on some occasion the controller gave an output a second time notably at  $t = 30s$ , indicating that the controller has yet to be fully tuned.

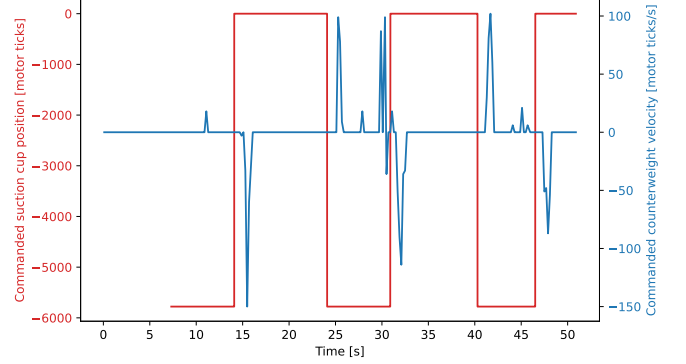


Fig. 11: Commanded suction cup position and Commanded counterweight velocity in time

### B. Software

1) *System response time evaluation:* This experiment measured the Retrieve Box operation's speed in the frontend and the backend. The objective was to determine how the system performs when retrieval requests are made. The retrieve operation was performed 10 times for randomly selected 5 boxes to ensure the results were consistent and accurate. The average response times for both the frontend and backend were calculated and plotted on a graph with the Box IDs. The results are shown in Figure 12. The orange bars show the frontend response times, while the gray bars show the backend response times. Frontend response times were consistently higher than backend response times across all box IDs. The retrieval time in frontend will naturally be higher than in the backend as it is the total time taken including the backend retrieval process but as seen in Figure 12, it is more than 2 times the backend response time. This disparity between the frontend and backend times hints at the fact that a large part of the retrieval time is spent in processing or communication in the frontend. Backend response times were steadier, with a rather linear processing. The frontend times varied more, probably due to network delays or different ways of displaying things.

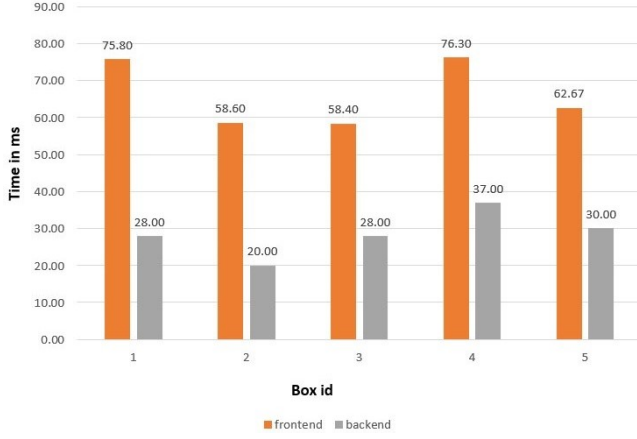


Fig. 12: System Response time

## VI. CONCLUSION AND FUTURE WORK

### A. Thrust and counterweight test

The test proved the hypothesis which means that changes have to be made to the platform. The hexacopter configuration is not the optimal one for this system. Propellers should be moved as far as possible from the tray. A better drone configuration would be X4 or - if more lift is required - X8 (motors on top and bottom of arms). They would reduce the disturbances of the airflow.

Currently the size of the deadzone is too large or the gain of the controller is too low, optimization of the counterweight control loop would allow the system to be more responsive and smoother leading to more stable flight.

### B. Software improvements and future work

The autonomous drone system's software architecture, which employs ROS, React, and PostgreSQL, establishes a robust basis for the project. Nevertheless, various improvements could greatly enhance its performance, scalability, and overall user experience.

As observed in the test results, while the backend is performing quite well with stable performance, fixing inefficiencies in the frontend could bring significant improvements to the whole system. Future work in this direction could involve deconstructing the frontend processes to identify inefficiencies or optimizing the data transmission between the frontend and backend components to reduce the total time taken.

Similarly, future research efforts can include database update functionalities. In the earlier stages of the project, there were discussions about automating database updates after a successful pick-and-place operation. The idea presented was, after a successful operation, the drone would send a command via a ROS topic to update the database with the latest information about the boxes. While this functionality was not implemented in the prototype, it can be done to make the database more accurate and reduce the number of manual updates.

## REFERENCES

- [1] Amr Abdelfattah, Tamer Abdelkader, and El-Sayed El-Horbarty. Ramws: Reliable approach using middleware and websockets in mobile cloud computing. *Ain Shams Engineering Journal*, 11, 06 2020. doi:10.1016/j.asej.2020.04.002.
- [2] Atharva Choudhari, Dnyanda Siddhapurmam, Tejas Umale, and Pranav Khadkotkar. Design and development of package delivery drone. 12:1009–1020, 06 2024.
- [3] Open Source Robotics Foundation. Ros 2 documentation: Humble, 2024. Accessed: 2024-12-20. URL: <https://docs.ros.org/en/humble/index.html>.
- [4] Open Source Robotics Foundation. roslib - ros wiki, 2024. Accessed: 2024-12-05. URL: <https://wiki.ros.org/roslib>.
- [5] Foxglove. Using rosbridge with ros2, 2024. Accessed: 2024-12-19. URL: <https://foxglove.dev/blog/using-rosbridge-with-ros2>.
- [6] Artem Ivanov, Aufar Zakiev, Tatyana Tsoy, and Kuo-Hsien Hsia. Online monitoring and visualization with ros and reactjs. pages 1–4, 2021. doi:10.1109/SIBCON50419.2021.9438890.
- [7] MathWorks. Robot operating system (ros) basic concepts, 2024. Accessed: 2024-12-19. URL: [https://www.mathworks.com/help/ros/gs/robot-operating-system-ros-basic-concepts\\_mw\\_85271ff3-e5c4-4998-9fbd-7ee81be97d9.html](https://www.mathworks.com/help/ros/gs/robot-operating-system-ros-basic-concepts_mw_85271ff3-e5c4-4998-9fbd-7ee81be97d9.html).
- [8] Satyam Pawale, Harish Kunder, P. Prajwal, D.J. Shriprasad, and S. Vinay. Warehouse inventory management: Drone-powered semi-automation with raspberry pi and network integration. 2024. Cited by: 0. URL: <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85205826843&doi=10.1109%2fCONECCT62155.2024.10677264&partnerID=40&md5=a05ba9ac1d3ab0371bf8a23a6d2fea49>, doi:10.1109/CONECCT62155.2024.10677264.
- [9] Silvia Proia, Graziana Cavone, Antonio Camposeo, Fabio Ceglie, Raffaele Carli, and Mariagrazia Dotoli. Safe and ergonomic human-drone interaction in warehouses. volume 2022-October, page 6681 – 6686, 2022. Cited by: 6. URL: <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85146319710&doi=10.1109%2fIROS47612.2022.9981469&partnerID=40&md5=61b20f98ff557ea77ff2ae551b6207b>, doi:10.1109/IROS47612.2022.9981469.
- [10] Nishant Sharma and Rekha Agarwal. *HTTP, WebSocket, and SignalR: A Comparison of Real-Time Online Communication Protocols*, pages 128–135. 09 2023. doi:10.1007/978-3-031-44084-7\_13.

| Task                                                                  | Contributors                                             |
|-----------------------------------------------------------------------|----------------------------------------------------------|
| Database definition and Setup                                         | Simone Cosentino, Nina Siwakoti                          |
| Integration of the database into ROS (ROS node creation)              | Nina Siwakoti, Simone Cosentino                          |
| ROSBridge server setup and integration                                | Simone Cosentino, Nina Siwakoti                          |
| Development of React application and integration with ROS environment | Simone Cosentino, Nina Siwakoti                          |
| Path planning algorithm                                               | Nina Siwakoti, Simone Cosentino, Jonas Mariager Jakobsen |
| Development of real time drone control                                | Jonas Mariager Jakobsen                                  |
| Development of hardware interface ROS nodes                           | Jonas Mariager Jakobsen, Adam Kolodziej                  |
| Design and development of drone construction and physical mechanisms  | Adam Kolodziej                                           |
| Design and development of electronics                                 | Adam Kolodziej                                           |
| Tests of physical systems                                             | Adam Kolodziej, Jonas Mariager Jakobsen                  |
| Creation of technical documentation                                   | All                                                      |

TABLE I: Individual contributions of team members.